
Inside the Model: The Feedback Loop

How work becomes heat and heat becomes slower work — how the job's timesteps line up with the telemetry clock — and how six scenarios push on that loop in six different places.

1 Hz

TELEMETRY INTERVAL

192 ms

ONE SOLVER TIMESTEP

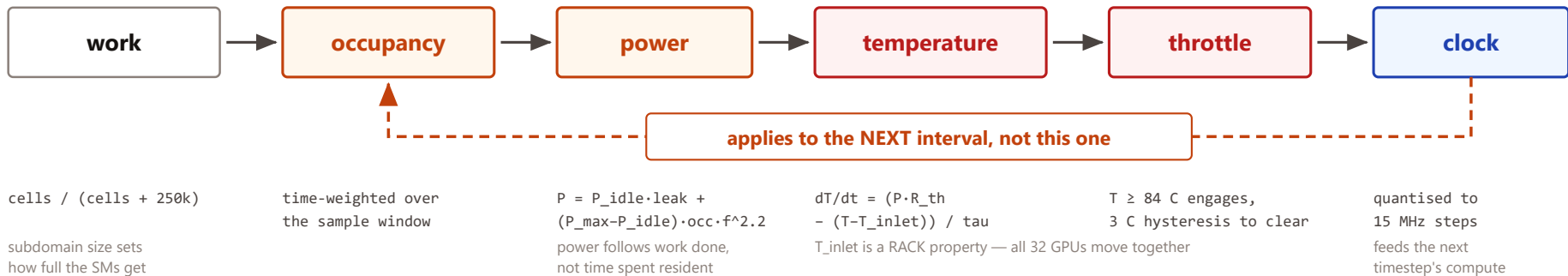
6

SCENARIOS, 6 SIGNATURES

THE LOOP

The only place an output feeds back into an input

Everything else in the simulator is a one-way function. This is the one cycle.



Why the lag matters

Power and temperature are evaluated with the clock that was in force *during* the interval; the governors then set the clock for the *next* one. Collapse that and the loop becomes an algebraic identity — you lose the ramp entirely.

Why it is self-limiting

A throttled GPU draws less power, so it heats its own rack inlet less. The loop settles instead of running away — but it overshoots slightly first, which is what real parts do.

Where faults enter

A cooling fault raises T_{inlet} . A RAS fault caps the clock directly. Neither touches telemetry — both are physical changes the loop then propagates on its own.

Power follows occupancy — not utilisation

Two channels that naive models treat as the same signal. Separating them is what makes a stall visible.

A rank blocked at the barrier



Both intervals have a kernel **resident**. Only one is doing anything.

utilization_pct

says nothing is wrong



sm_occupancy_pct

the stall is here



power_w

follows occupancy, so it falls too



Illustrative window: 62% of the interval computing at 87% occupancy, 38% spinning at 5%. Bars are the model's own formulas on those inputs.

Utilisation is near-flat by design

It measures whether *any* kernel is resident. The collective spins rather than blocking, so a waiting rank still reports ~99% busy. In all six scenarios it barely moves.

Occupancy carries the signal

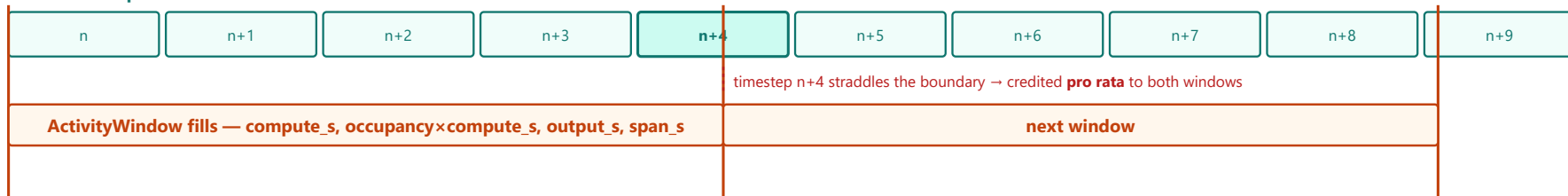
Time-weighted over the window, so waiting drags it down. It is the channel that separates a **synchronisation stall** from a genuinely idle device — and it drives power, which drives the whole loop.

TWO CLOCKS

How the job's timesteps meet the 1 Hz telemetry interval

The solver runs on its own rhythm. The loop advances once per telemetry tick — roughly every five timesteps.

solver timesteps — ~192 ms each



tick t

tick t+1s

tick t+2s

at the tick, the window drains:

`compute_fraction (cf) = compute_s / span` share of the second computing
`occupancy = occupancy_weighted / compute_s` average WHILE computing
`occ_active = cf * occupancy + (1 - cf) * 0.05` whole second → sm_occupancy_pct

`governor.step(dt = span)`

one pass round the loop:
power → temperature → clock

new clock_factor

governs timesteps n+5 on,
until the next tick

Roughly 5 timesteps per sample

At the medium mesh a timestep is ~192 ms and the interval is 1 s. The loop therefore updates about **once every five timesteps** — not once per timestep.

1 Hz is a deliberate limit

Real fleets poll DCGM at about 1 Hz, so a stall shorter than a second is genuinely invisible — a limitation of the model *and* of real monitoring.

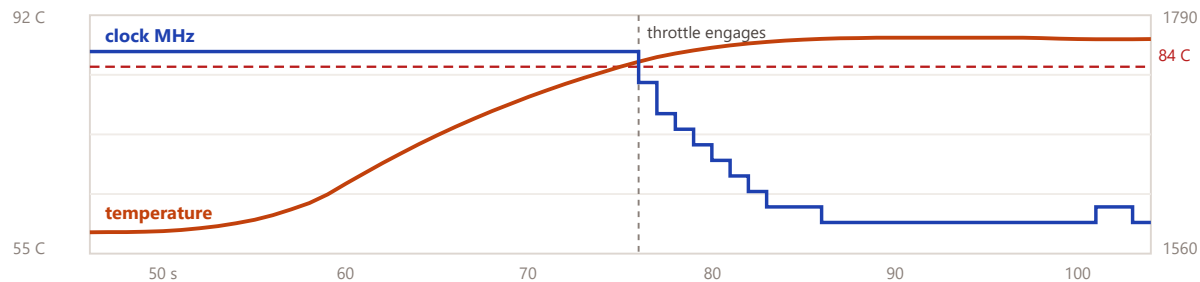
Two different grains on disk

`telemetry_*` tables carry ~202 rows per run at 1 Hz. `rank_` and `job_performance` carry 1000 — one per timestep. The windows are not the same windows.

THE LOOP IN ACTION

A cooling fault, from injection to settled staircase

Real telemetry, one GPU in rack r1, `thermal` scenario at the medium mesh.



Cooling drops to 30% at iteration 250. Inlet climbs, the die follows with a 20 s lag, crosses 84 C at $t = 76$ s, and the governor steps the clock down 15 MHz at a time until the loop settles.

Where the model stops. A more severe version of this is a real operational failure: in a hot summer, an air-conditioning fault in the machine room can drive room and cluster temperature up until machines **enter thermal protection and shut down**, requiring manual intervention. This simulator models the earlier stage of that chain — cooling degrades, inlet and die temperature rise, throttling cuts clock and power, and the system settles at a new equilibrium. The emergency-shutdown state is deliberately not modelled.

	before	after
Temperature	58.3 C	88.5 C
Clock	1755	1590 MHz
Power	572 W	480 W
Timestep	191.5 ms	208.1 ms

Ramp, then staircase

The 20 s thermal lag turns a step change in cooling into a **gradual ramp**; discrete clock bins turn the response into a **staircase** — eleven 15 MHz steps.

Because $P \propto f^{2.2}$ power falls faster than clock — **the two falling together is the throttling signature**.

THE SCENARIOS

Six faults, six different entry points into the loop

Each injection changes **physical state** only. Nothing writes to telemetry — every signal is a downstream consequence.

Scenario	Truth	What is injected	Where it enters	How it reaches the job
healthy	no fault	nothing	—	baseline: ragged partition only
thermal	rack	CRAC to 30% over 60 steps	T_inlet	inlet → temperature → throttle → clock
gpu_degradation	gpu	HBM to 85%, RAS clock cap 90%	clock cap + mem bw	bypasses temperature — enters at the governor
straggler	rank	26 episodes on 3 GPUs, 55–80%	SM share	outside the loop: no counter moves
network_domain	rack	7 of 8 uplinks down, 2% error rate	fabric	outside the loop: halo slows, occupancy falls
phase_change	no fault	output every 20 steps, 4× bytes	workload	legitimate: storage and I/O, every rank alike

Two enter the loop

thermal at the inlet, **gpu_degradation** at the governor — the only two that throttle.

Two are invisible to it

straggler steals SM time; **network_domain** slows the wire.

Two are not faults

Flagging **phase_change** fails as surely as missing the thermal fault.

Why the straggler is in the set — it was observed, not invented. On a production HPC cluster, MPI ranks occasionally outlive the job that spawned them and must be killed by hand; a new job landing on those nodes then contends with them. This is the GPU analogue: intermittent loss of SM throughput, clock untouched.

THE SIGNATURES

What each scenario actually moves in the telemetry

Early window against late window, medium mesh. This is the whole basis for telling them apart.

Scenario	iteration time	rank spread	occupancy	power	peak temp	throttled	link errors	utilisation
healthy	+0.3%	+0.8%	+1.3%	+2.3%	59 C	0%	0	99.9%
straggler	+1.3%	+6.4%	-0.3%	+0.4%	58 C	0%	0	99.9%
network_domain	+19.8%	+384%	-14.2%	-11.6%	54 C	0%	7.9 M	99.9%
thermal	+8.9%	+176%	-4.1%	-6.8%	88.7 C	15.7%	0	99.9%
gpu_degradation	+26.7%	+547%	-18.6%	-15.6%	53 C	0.5%	0	99.9%
phase_change	+10.1%	+0.8%	-7.0%	-5.2%	57 C	0%	0	99.9%

The straggler hides

It moves **one** channel — rank spread, by 6.4%. Every hardware counter says the fleet is healthy, because it is.

Spread is the discriminator

phase_change slows the job 10% with spread **flat** — every rank affected identically, which no localised fault can do.

Utilisation never moves

99.9% in all six columns. A stack watching only utilisation would see nothing, ever.

THE DIAGNOSIS

Six ordered checks, first match wins

A strict `if / return` chain over telemetry only — it never sees the scenario's label.

Check	Returns
A any sample throttled?	<code>HARDWARE_FAULT / rack</code> if <code>THERMAL</code> <code>HARDWARE_FAULT / gpu</code> if <code>RELIABILITY</code>
B one rack drifted > 8 C from its peers?	<code>HARDWARE_FAULT / rack</code> · medium
C link down, or errors / drops > 1000?	<code>HARDWARE_FAULT / rack</code> · high
D any rank pacing the barrier ≥ 2% of steps?	<code>HARDWARE_FAULT / rank</code> · medium
E slowdown < 3%?	<code>NOMINAL / none</code> · high
F no test — unconditional fallback	<code>WORKLOAD_CHANGE / workload</code>

Gate B exists because a cooling fault on a light workload never crosses 84 C — on the coarse mesh the same fault throttles nothing at all, and only the cross-rack drift catches it.

Scenario	gate	verdict
healthy	E	<code>NOMINAL</code>
straggler	D	<code>HW_FAULT / rank</code>
network_domain	C	<code>HW_FAULT / rack</code>
thermal	A	<code>HW_FAULT / rack</code>
gpu_degradation	A	<code>HW_FAULT / gpu</code>
phase_change	F	<code>WORKLOAD_CHANGE</code>

Hardware is checked before impact

Gates A–D all run *before* the slowdown gate. A degraded link can cost almost no wall time while its counters scream — calling that nominal is how a fleet ends up running on a quietly failing fabric.

IN SHORT

One loop, two clocks, six signatures

The loop

work → occupancy → power →
temperature → throttle → clock, closing
back on the next interval.

The one-interval lag is what turns a step change in cooling into a ramp and then a staircase — and what stops the loop being an algebraic identity.

The two clocks

The solver advances every ~192 ms; the loop advances once per **1 Hz** tick. Work accumulates in a window, pro rata across boundaries, and drains at the tick.

The clock set at one tick governs every timestep until the next.

The signatures

Faults enter at four different points — inlet, governor, SM share, fabric — and each leaves a distinct set of channels moving.

Six ordered checks read those channels back and name the tier. Hardware first, impact last.

The one-line version. Nothing in the model writes a fault into telemetry. Every signal a detector reads is a physical consequence, one interval late — which is exactly what makes the diagnosis worth scoring.